

Cookbook: Local Apache Pulsar Cluster with Kubernetes Functions Runtime

Overview

This cookbook walks you through standing up a local Apache Pulsar 4.0.x cluster on Docker with Zookeeper, BookKeeper, Broker, Proxy, and a Pulsar Functions Worker configured to run functions on Kubernetes via Minikube. It also sets up Envoy/Contour using the Kubernetes Gateway API to expose gRPC endpoints for functions, configures DNS for service discovery, installs Kyverno to auto-generate GRPCRoutes, and demonstrates deploying and testing a sample function.

i Tested with Pulsar images 4.0.10 and Minikube k8s v1.34.0. The guide assumes Linux, Docker, kubectl, helm, yq, jq, and a helper alias ds for docker-compose-like commands. Substitute ds with docker where applicable.

- [Overview](#)
- [Prerequisites](#)
- [Architecture at a Glance](#)
- [Step-by-Step Setup](#)
 - [1\) Initialize environment and pull images](#)
 - [2\) Reset Minikube and Docker network](#)
 - [3\) Start Minikube for the functions runtime](#)
 - [4\) Prepare Pulsar conf and start Zookeeper](#)
 - [5\) Start BookKeeper](#)
 - [6\) Start Broker](#)
 - [7\) Install Gateway API CRDs, Contour, and Gateway](#)
 - [8\) Local DNS for functions domain](#)
 - [9\) Kubeconfig for Functions Worker](#)
 - [10\) Configure and start Functions Worker \(Kubernetes runtime\)](#)
 - [11\) Configure and start Pulsar Proxy](#)
 - [12\) Host function artifacts with nginx](#)
 - [13\) Install Kyverno and grant Gateway API access](#)
 - [14\) Optional: Patch function services for metrics](#)
 - [15\) Kyverno policy to auto-generate GRPCRoute per function](#)
 - [16\) Deploy and test a sample function](#)
- [Verification and Troubleshooting](#)
 - [Verify network and container IPs](#)
 - [Common issues](#)
- [Teardown and Reset](#)
- [Reference Commands \(Quick Copy\)](#)
- [Appendix: Rationale and Notes](#)

Prerequisites

- Docker Engine installed and running

- kubectl, minikube, helm, jq, yq installed
- Ports available on host: 18080 (proxy), 16650 (proxy), Minikube NodePort range
- At least 8 GB RAM for Minikube profile
- Optional: nginx Docker image to serve function JARs

Architecture at a Glance

- Local Docker network simulating Pulsar control plane: ZK, BK, Broker, Proxy, Functions Worker
- Minikube cluster hosts Kubernetes runtime for Pulsar Functions jobs
- Contour Gateway (Envoy) provides Gateway API support; Kyverno auto-creates GRPCRoutes per function service
- dnsmasq container resolves a convenience domain to Minikube node IP

Step-by-Step Setup

1) Initialize environment and pull images

```

1 # Pull Pulsar images
2 docker pull apachepulsar/pulsar:4.0.10
3 docker pull apachepulsar/pulsar-all:4.0.10
4
5 # Clean any previous run
6 sudo rm -rf conf-pulsar/ data/ conf-sb1/ pulsar-ca/
7
8 # Variables
9 IMAGE="apachepulsar/pulsar-all:4.0.10"
10 CONF_DIR="pulsar-conf"
11 IP_ADDR="$(hostname -I | awk '{print $1}')" # routable IP (not
12 loopback)
13 ZK1="${IP_ADDR}"
14 BK1="${IP_ADDR}"
15 BRK1="${IP_ADDR}"
16 FWRK="${IP_ADDR}"
17 PRXY="${IP_ADDR}"
18 CLUSTER="pulsar-local"

```

Ensure IP_ADDR is reachable from inside Docker containers (e.g., your primary LAN IP or Minikube IP as needed). Avoid 127.0.0.1.

2) Reset Minikube and Docker network

```

1 minikube stop --profile=k8s-pulsar-functions
2 minikube delete --profile=k8s-pulsar-functions
3
4 # Stop and clean prior containers and network (ignore errors if not
5 present)
6 docker stop ZK1 BK1 BRK1 FWRK PRXY nginx pulsar-dns socat-proxy || true
7 docker network rm k8s-pulsar-functions || true

```

3) Start Minikube for the functions runtime

```

1 minikube start \
2   --cni calico \
3   --profile=k8s-pulsar-functions \
4   --apiserver-names=k8s-pulsar-functions,*.k8s-pulsar-functions \
5   --kubernetes-version=v1.34.0 \
6   --memory=8000 \
7   --cpus=2 \
8   --nodes=1
9
10 minikube profile k8s-pulsar-functions

```

4) Prepare Pulsar conf and start Zookeeper

```

1 # Initialize local Pulsar configuration directory
2 sudo rm -rf data ${CONF_DIR}
3 docker run --rm --user=0:0 -v "${PWD}:/host" ${IMAGE} cp -r
4   "/pulsar/conf" "/host/${CONF_DIR}"
5
6 # Configure Zookeeper
7 echo "server.1=${ZK1}:2888:3888" | sudo tee -a
8   ${CONF_DIR}/zookeeper.conf
9 mkdir -p data/zookeeper
10 echo 1 > data/zookeeper/myid
11
12 # Create a docker network to host components (same name as Minikube IP
13 subnet if desired)
14 docker network create --subnet=192.168.49.0/24 k8s-pulsar-functions ||
15   true
16
17 # Run Zookeeper
18 docker run --rm -d --user=0:0 --name=ZK1 --network=k8s-pulsar-
19   functions \
20   -v "${PWD}/${CONF_DIR}:/pulsar/conf" -v "${PWD}/data:/pulsar/data" \
21   ${IMAGE} bin/pulsar zookeeper
22
23 sleep 1
24
25 # Initialize cluster metadata
26 docker exec -it ZK1 bin/pulsar initialize-cluster-metadata \
27   --cluster ${CLUSTER} \
28   --zookeeper ZK1:2181 \
29   --configuration-store ZK1:2181 \
30   --web-service-url http://BRK1:8080 \
31   --web-service-url-tls https://BRK1:8443 \
32   --broker-service-url pulsar://BRK1:6650 \
33   --broker-service-url-tls pulsar+ssl://BRK1:6651

```

5) Start BookKeeper

```

1 # Point BookKeeper to ZK and relax for single node; expose Prometheus
2 port
3 sudo sed -i -e "s/^zkServers=.*zkServers=ZK1:2181/"
4   ${CONF_DIR}/bookkeeper.conf
5 sudo sed -i -e "s/^allowLoopback=.*allowLoopback=true/"
6   ${CONF_DIR}/bookkeeper.conf
7 sudo sed -i -e
8   "s/^prometheusStatsHttpPort=.*prometheusStatsHttpPort=8001/"
9   ${CONF_DIR}/bookkeeper.conf
10
11 docker run --rm -d --user=0:0 --name=BK1 --network=k8s-pulsar-functions
12 \
13   -v "${PWD}/${CONF_DIR}:/pulsar/conf" -v "${PWD}/data:/pulsar/data" \
14   ${IMAGE} bin/bookkeeper bookie

```

6) Start Broker

```
1 # Single-node broker config
2 echo "superUserRoles=superuser,admin,proxy" | sudo tee -a
   ${CONF_DIR}/broker.conf
3 sudo sed -i -e "s/^zookeeperServers=.*\/zookeeperServers=ZK1:2181/"
   ${CONF_DIR}/broker.conf
4 sudo sed -i -e
   "s/^configurationStoreServers=.*\/configurationStoreServers=ZK1:2181/"
   ${CONF_DIR}/broker.conf
5 sudo sed -i -e "s/^clusterName=.*\/clusterName=${CLUSTER}/"
   ${CONF_DIR}/broker.conf
6 sudo sed -i -e
   "s/^managedLedgerDefaultEnsembleSize=.*\/managedLedgerDefaultEnsembleSi
   ze=1/" ${CONF_DIR}/broker.conf
7 sudo sed -i -e
   "s/^managedLedgerDefaultWriteQuorum=.*\/managedLedgerDefaultWriteQuorum
   =1/" ${CONF_DIR}/broker.conf
8 sudo sed -i -e
   "s/^managedLedgerDefaultAckQuorum=.*\/managedLedgerDefaultAckQuorum=1/"
   ${CONF_DIR}/broker.conf
9 sudo sed -i -e
   "s/^functionsWorkerEnabled=.*\/functionsWorkerEnabled=false/"
   ${CONF_DIR}/broker.conf
10 sudo sed -i -e "s/^proxyRoles=.*\/proxyRoles=proxy/"
   ${CONF_DIR}/broker.conf
11
12 docker run --rm -d --user=0:0 --name=BRK1 --network=k8s-pulsar-
   functions \
13   -v "${PWD}/${CONF_DIR}:/pulsar/conf" -v "${PWD}/data:/pulsar/data" \
14   ${IMAGE} bin/pulsar broker
```

7) Install Gateway API CRDs, Contour, and Gateway

```
1 # Gateway API CRDs
2 kubectl apply -k "https://github.com/kubernetes-sigs/gateway-
   api/config/crd?ref=v1.1.0"
3 kubectl apply -f https://raw.githubusercontent.com/kubernetes-
   sigs/gateway-
   api/v1.1.0/config/crd/experimental/gateway.networking.k8s.io_tcproutes
   .yaml
4
5 # Contour + Gateway Provisioner
6 kubectl apply -f https://projectcontour.io/quickstart/contour.yaml
7 kubectl apply -f https://projectcontour.io/quickstart/contour-gateway-
   provisioner.yaml
8 kubectl get pods -n projectcontour
9
10 # Create namespace and Gateway
11 kubectl create ns pulsar-functions-ns || true
12 cat << 'EOF' | kubectl apply -f -
13 kind: GatewayClass
14 apiVersion: gateway.networking.k8s.io/v1
15 metadata:
16   name: contour
17 spec:
18   controllerName: projectcontour.io/gateway-controller
19 EOF
20
21 cat << 'EOF' | kubectl apply -f -
22 kind: Gateway
23 apiVersion: gateway.networking.k8s.io/v1
24 metadata:
25   name: pulsar-functions-gateway
26   namespace: pulsar-functions-ns
```

```

27 spec:
28   gatewayClassName: contour
29   listeners:
30     - name: grpc-functions
31       protocol: HTTP
32       port: 80
33 EOF
34
35 # Determine the NodePort exposed by Envoy for the Gateway
36 kubectl get svc envoy-pulsar-functions-gateway -n pulsar-functions-ns
37 -o yaml | yq '.spec.ports[] | select(.protocol == "TCP") | .nodePort'
38 GATEWAY_NODE_PORT_FW=$(kubectl get svc envoy-pulsar-functions-gateway
39 -n pulsar-functions-ns -o yaml | yq '.spec.ports[] | select(.protocol
40 == "TCP") | .nodePort')

```

- Contour's Gateway Provisioner creates an Envoy Service (type LoadBalancer/NodePort in Minikube). We reuse its NodePort for the Functions Worker gRPC configuration.

8) Local DNS for functions domain

```

1 # Map a friendly domain to Minikube's node IP
2 cat > /tmp/dnsmasq.conf <<'EOF'
3 address=/pulsar-functions.midgard.acme.com/192.168.49.2
4 EOF
5
6 docker run --rm -d --name=pulsar-dns --network=k8s-pulsar-functions \
7   --ip=192.168.49.10 --cap-add=NET_ADMIN \
8   -v /tmp/dnsmasq.conf:/etc/dnsmasq.conf andyshinn/dnsmasq:2.78

```

9) Kubeconfig for Functions Worker

```

1 sudo mkdir -p ${CONF_DIR}/k8s/
2 cat <<EOF | sudo tee ${CONF_DIR}/k8s/config
3 apiVersion: v1
4 clusters:
5   - cluster:
6       certificate-authority: /pulsar/conf/k8s/ca.crt
7       name: k8s-pulsar-functions
8       server: https://k8s-pulsar-functions:8443
9 contexts:
10  - context:
11      cluster: k8s-pulsar-functions
12      namespace: default
13      user: k8s-pulsar-functions
14      name: k8s-pulsar-functions
15 current-context: k8s-pulsar-functions
16 kind: Config
17 preferences: {}
18 users:
19   - name: k8s-pulsar-functions
20     user:
21       client-certificate: /pulsar/conf/k8s/client.crt
22       client-key: /pulsar/conf/k8s/client.key
23 EOF
24
25 # Copy Minikube certs into the config directory
26 sudo cp ~/.minikube/ca.crt ${CONF_DIR}/k8s/
27 sudo cp ~/.minikube/profiles/k8s-pulsar-functions/client.crt
28 ${CONF_DIR}/k8s/
29 sudo cp ~/.minikube/profiles/k8s-pulsar-functions/client.key
30 ${CONF_DIR}/k8s/
31 sudo openssl x509 -in ${CONF_DIR}/k8s/ca.crt -text -noout

```

10) Configure and start Functions Worker (Kubernetes runtime)

```
1 # Functions Worker core endpoints
2 sudo sed -i -e "s/^workerId:./workerId: FWRK/"
   ${CONF_DIR}/functions_worker.yml
3 sudo sed -i -e "s/^workerHostname:./workerHostname: FWRK/"
   ${CONF_DIR}/functions_worker.yml
4 sudo sed -i -e
   "s/^configurationStoreServers:./configurationStoreServers: ZK1:2181/"
   ${CONF_DIR}/functions_worker.yml
5 sudo sed -i -e "s/^pulsarFunctionsCluster:./pulsarFunctionsCluster:
   ${CLUSTER}/" ${CONF_DIR}/functions_worker.yml
6 sudo sed -i -e "s|^pulsarServiceUrl:.*|pulsarServiceUrl:
   pulsar://BRK1:6650|" ${CONF_DIR}/functions_worker.yml
7 sudo sed -i -e "s|^pulsarWebServiceUrl:.*|pulsarWebServiceUrl:
   http://BRK1:8080|" ${CONF_DIR}/functions_worker.yml
8 sudo sed -i -e
   "s|^configurationMetadataStoreUrl:.*|configurationMetadataStoreUrl:
   zk:ZK1:2181|" ${CONF_DIR}/functions_worker.yml
9
10 # Switch to Kubernetes runtime factory
11 sudo sed -i -e
   '/^functionRuntimeFactoryClassName:./,/extraFunctionDependenciesDir:/
   s/^/# /' ${CONF_DIR}/functions_worker.yml
12 sudo sed -i -e
   "s|^#functionRuntimeFactoryClassName:.*KubernetesRuntimeFactory|functi
   onRuntimeFactoryClassName:
   org.apache.pulsar.functions.runtime.kubernetes.KubernetesRuntimeFactor
   y|" ${CONF_DIR}/functions_worker.yml
13 sudo sed -i -e
   "s|^#functionRuntimeFactoryConfigs:.*|functionRuntimeFactoryConfigs:|"
   ${CONF_DIR}/functions_worker.yml
14 sudo sed -i -e "s|^#    k8Uri:.*|    k8Uri: null|"
   ${CONF_DIR}/functions_worker.yml
15 sudo sed -i -e "s|^#    jobNamespace:.*|    jobNamespace: \"pulsar-
   functions-ns\"|" ${CONF_DIR}/functions_worker.yml
16 sudo sed -i -e "s|^#    pulsarDockerImageName:.*|
   pulsarDockerImageName: \"apache/pulsar:4.0.10\"|"
   ${CONF_DIR}/functions_worker.yml
17 sudo sed -i -e "s|^#    pulsarAdminUrl:.*|    pulsarAdminUrl:
   http://FWRK:6750|" ${CONF_DIR}/functions_worker.yml
18 sudo sed -i -e "s|^#    pulsarServiceUrl:.*|    pulsarServiceUrl:
   pulsar://BRK1:6650|" ${CONF_DIR}/functions_worker.yml
19 sudo sed -i -e "s|^#    grpcPort:.*|    grpcPort:
   ${GATEWAY_NODE_PORT_FW}|" ${CONF_DIR}/functions_worker.yml
20 sudo sed -i -e "s|^#    metricsPort:.*|    metricsPort: 9094|"
   ${CONF_DIR}/functions_worker.yml
21 sudo sed -i -e "s|^#    submittingInsidePod:.*|    submittingInsidePod:
   false|" ${CONF_DIR}/functions_worker.yml
22
23 # TLS convenience (insecure for local)
24 sudo sed -i -e "s|^tlsTrustStore:.*|tlsTrustStore:
   /pulsar/conf/auth/functionWorkerTrustStore.jks|"
   ${CONF_DIR}/functions_worker.yml
25 sudo sed -i -e "s|^tlsTrustStorePassword:.*|tlsTrustStorePassword:
   changeit|" ${CONF_DIR}/functions_worker.yml
26 sudo sed -i -e
   "s|^tlsAllowInsecureConnection:.*|tlsAllowInsecureConnection: true|"
   ${CONF_DIR}/functions_worker.yml
27
28 # Allow worker to fetch JARs from local nginx
29 echo "additionalEnabledFunctionsUrlPatterns:" | sudo tee -a
   ${CONF_DIR}/functions_worker.yml
30 echo " - \"http://nginx/.*/\" | sudo tee -a
   ${CONF_DIR}/functions_worker.yml
31
```

```

32 # requires a build of pulsar fork
   https://github.com/iantowey/pulsar/tree/feature/parameterise-k8s-
   service-domain-suffix with required config
33 # set Kubernetes service domain suffix used by worker (helps DNS
   resolution)
34 echo "kubernetesServiceDomainSuffix: \"pulsar-
   functions.midgard.acme.com\"" | sudo tee -a
   ${CONF_DIR}/functions_worker.yml
35
36
37
38 # Start Functions Worker (include runtime jar if needed)
39 docker run --rm -d --user=0:0 --name=FWRK --network=k8s-pulsar-
   functions \
40   --dns=192.168.49.10 --dns=8.8.8.8 \
41   -e "KUBECONFIG=/pulsar/conf/k8s/config" \
42   -e "JAVA_TOOL_OPTIONS=-
   Djdk.internal.httpclient.disableHostnameVerification=true" \
43   -v "${PWD}/${CONF_DIR}:/pulsar/conf" -v "${PWD}/data:/pulsar/data" \
44   -v "$HOME/.m2/repository/org/apache/pulsar/pulsar-functions-
   runtime/4.0.10/pulsar-functions-runtime-
   4.0.10.jar:/pulsar/lib/org.apache.pulsar-pulsar-functions-runtime-
   4.0.10.jar" \
45   ${IMAGE} bin/pulsar functions-worker

```

11) Configure and start Pulsar Proxy

```

1 echo "brokerProxyAllowedTargetPorts=6650,6651" | sudo tee -a
   ${CONF_DIR}/proxy.conf
2 echo "authenticateMetricsEndpoint=false" | sudo tee -a
   ${CONF_DIR}/proxy.conf
3 sudo sed -i -e
   "s|^authenticationProviders=.*|authenticationProviders=org.apache.puls
   ar.broker.authentication.AuthenticationProviderToken|"
   ${CONF_DIR}/proxy.conf
4 sudo sed -i -e
   "s|^brokerServiceURL=.*|brokerServiceURL=pulsar://BRK1:6650|"
   ${CONF_DIR}/proxy.conf
5 sudo sed -i -e
   "s|^brokerServiceURLTLS=.*|brokerServiceURLTLS=pulsar+ssl://BRK1:6651|
   " ${CONF_DIR}/proxy.conf
6 sudo sed -i -e
   "s|^brokerWebServiceURL=.*|brokerWebServiceURL=http://BRK1:8080|"
   ${CONF_DIR}/proxy.conf
7 sudo sed -i -e
   "s|^brokerWebServiceURLTLS=.*|brokerWebServiceURLTLS=https://BRK1:8443
   |" ${CONF_DIR}/proxy.conf
8 sudo sed -i -e
   "s|^configurationStoreServers=.*|configurationStoreServers=ZK1:2181|"
   ${CONF_DIR}/proxy.conf
9 sudo sed -i -e
   "s|^functionWorkerWebServiceURL=.*|functionWorkerWebServiceURL=http://
   FWRK:6750|" ${CONF_DIR}/proxy.conf
10 sudo sed -i -e
   "s|^functionWorkerWebServiceURLTLS=.*|functionWorkerWebServiceURLTLS=h
   ttps://FWRK:6751|" ${CONF_DIR}/proxy.conf
11 sudo sed -i -e "s|^zookeeperServers=.*|zookeeperServers=ZK1:2181|"
   ${CONF_DIR}/proxy.conf
12 sudo sed -i -e "s|^clusterName=.*|clusterName=${CLUSTER}|"
   ${CONF_DIR}/proxy.conf
13 sudo sed -i -e "s|^webServicePort=.*|webServicePort=18080|"
   ${CONF_DIR}/proxy.conf
14 sudo sed -i -e "s|^servicePort=.*|servicePort=16650|"
   ${CONF_DIR}/proxy.conf
15 sudo sed -i -e
   "s|^superUserRoles=.*|superUserRoles=superuser,admin,proxy|"

```



```

16 ${CONF_DIR}/proxy.conf
17 docker run --rm -d --user=0:0 --name=PRXY \
18 -p 18080:18080 -p 16650:16650 \
19 --network=k8s-pulsar-functions \
20 -v "${PWD}/${CONF_DIR}/pulsar/conf" -v "${PWD}/data:/pulsar/data" \
21 ${IMAGE} bin/pulsar proxy

```

12) Host function artifacts with nginx

```

1 docker run --rm -d --name=nginx --network=k8s-pulsar-functions
  nginx:1.19.4
2
3 # Copy your function jar(s) into nginx html root
4 docker cp /path/to/pass-through-function-1.9.174-SNAPSHOT-java17-jar-
  with-dependencies.jar nginx:/usr/share/nginx/html
5 docker cp /path/to/pass-through-function-1.9.174-SNAPSHOT-java17-jar-
  with-dependencies.jar nginx:/usr/share/nginx/html/pass-through-
  function.jar
6
7 docker exec nginx ls -al /usr/share/nginx/html/

```

13) Install Kyverno and grant Gateway API access

```

1 helm repo add kyverno https://kyverno.github.io/kyverno/
2 helm repo update
3 helm install kyverno kyverno/kyverno -n kyverno --create-namespace
4
5 # ClusterRole for Kyverno to manage GRPCRoutes
6 cat << 'EOF' | kubectl apply -f -
7 apiVersion: rbac.authorization.k8s.io/v1
8 kind: ClusterRole
9 metadata:
10   name: kyverno-gateway-api-access
11   labels:
12     rbac.kyverno.io/aggregate-to-admission-controller: "true"
13     rbac.kyverno.io/aggregate-to-background-controller: "true"
14     rbac.kyverno.io/aggregate-to-reports-controller: "true"
15 rules:
16 - apiGroups: ["gateway.networking.k8s.io"]
17   resources: ["grpcroutes"]
18   verbs: ["create", "delete", "get", "list", "patch", "update", "watch"]
19 EOF

```

14) Optional: Patch function services for metrics

```

1 # If you maintain a patch manifest for Services to expose metrics,
  apply it:
2 # kubectl apply -n pulsar-functions-ns -f /tmp/patch-function-svc-for-
  metrics.yaml

```

15) Kyverno policy to auto-generate GRPCRoute per function

```

1 cat > /tmp/pulsar-kyverno-policy.yaml <<'EOF'
2 apiVersion: kyverno.io/v1
3 kind: ClusterPolicy
4 metadata:
5   name: auto-generate-pulsar-grpc-route
6 spec:
7   rules:
8   - name: generate-pulsar-grpc-route
9     match:

```



```

10     any:
11     - resources:
12         kinds: ["Service"]
13         namespaces: ["pulsar-functions-ns"]
14     preconditions:
15     all:
16     - key: "{{ request.object.metadata.name }}"
17       operator: Matches
18       value: "pf-.*"
19     generate:
20     apiVersion: gateway.networking.k8s.io/v1
21     kind: GRPCRoute
22     name: "{{ request.object.metadata.name }}-route"
23     namespace: pulsar-functions-ns
24     synchronize: true
25     data:
26     spec:
27     parentRefs:
28     - name: pulsar-functions-gateway
29     hostnames:
30     - "*.{{ request.object.metadata.name }}.pulsar-functions-
ns.pulsar-functions.midgard.acme.com"
31     rules:
32     - backendRefs:
33     - name: "{{ request.object.metadata.name }}"
34       port: "{{ request.object.spec.ports[0].port }}"
35 EOF
36
37 kubectl apply -f /tmp/pulsar-kyverno-policy.yaml

```

-  This policy watches Services named like pf-*. When a Pulsar Function deployment creates a matching Service, Kyverno generates a GRPCRoute pointing to it and binds to the configured Gateway.

16) Deploy and test a sample function

```

1  # Use a compatible pulsar-admin client; set PULSAR_ADMIN_URL to Proxy
  or Broker web service
2  export PULSAR_ADMIN_URL="http://localhost:18080"
3
4  # Create function (reads from pass-through1-in, writes to pass-
  through1-out)
5  pulsar-admin functions create \
6  --tenant public \
7  --namespace default \
8  --name pass-through1 \
9  --inputs persistent://public/default/pass-through1-in \
10 --output persistent://public/default/pass-through1-out \
11 --parallelism 1 \
12 --jar http://nginx/pass-through-function.jar \
13 --className com.acme.dataeng.FuncPantry.PassThroughFunction
14
15 # Check function status
16 pulsar-admin functions status --fqfn public/default/pass-through1 | jq
17 .
18
19 # Produce and consume to verify
20 pulsar-client consume -n 0 -s sub persistent://public/default/pass-
  through1-out
21 pulsar-client produce persistent://public/default/pass-through1-in -m
  '{"id":1,"name":"ian"}' -s ';'
22
23 # Cleanup

```

Verification and Troubleshooting

Verify network and container IPs

```

1 docker network inspect k8s-pulsar-functions | jq -c '.[].Containers[]
  | {host:.Name,ip:.IPv4Address}'
2 # Example output:
3 # {"host":"pulsar-dns","ip":"192.168.49.10/24"}
4 # {"host":"PRXY","ip":"192.168.49.7/24"}
5 # {"host":"k8s-pulsar-functions","ip":"192.168.49.2/24"}
6 # {"host":"nginx","ip":"192.168.49.8/24"}
7 # {"host":"FWRK","ip":"192.168.49.6/24"}
8 # {"host":"BK1","ip":"192.168.49.4/24"}
9 # {"host":"BRK1","ip":"192.168.49.5/24"}
10 # {"host":"ZK1","ip":"192.168.49.3/24"}

```

Common issues

- Functions stuck in Pending: check KUBECONFIG mounts, Namespace pulsar-functions-ns exists, worker has k8Uri null (uses in-cluster) with proper client certs.
- Cannot fetch JAR: confirm nginx is reachable from worker, additionalEnabledFunctionsUrlPatterns allows http://nginx/*
- gRPC unreachable: verify Envoy Service exists, Gateway listener on port 80, and GATEWAY_NODE_PORT_FW used by worker. In Minikube, curl nodeIP:NodePort from host.
- Broker metadata init fails: ensure ZK is up and reachable at ZK1:2181 and the initialize-cluster-metadata step runs only once.
- Admin/Client errors against Broker: use Proxy endpoints on localhost:18080 (web) and 16650 (binary), or map hostnames in /etc/hosts.

Teardown and Reset

```

1 # Stop containers
2 docker stop ZK1 BK1 BRK1 FWRK PRXY nginx pulsar-dns || true
3
4 # Remove network
5 docker network rm k8s-pulsar-functions || true
6
7 # Reset Minikube profile
8 minikube stop --profile=k8s-pulsar-functions
9 minikube delete --profile=k8s-pulsar-functions
10
11 # Clean local data
12 sudo rm -rf data ${CONF_DIR}

```

Reference Commands (Quick Copy)

- Proxy web: http://localhost:18080
- Proxy binary: pulsar://localhost:16650

- Gateway NodePort lookup: `kubectl get svc envoy-pulsar-functions-gateway -n pulsar-functions-ns -o yaml | yq '.spec.ports[] | select(.protocol == "TCP") | .nodePort'`

Appendix: Rationale and Notes

- We run Pulsar core services in Docker for simplicity while delegating function execution to Kubernetes for parity with production-like orchestration.
- Contour implements the Gateway API; Kyverno reduces manual GRPCRoute management overhead.
- The single-node ledger/broker settings are for local development only; do not use in production.